Let me be clear: using treaps as both trees and heaps is possible, perfectly legal, and it can even offer decent performance, under certain conditions, although, in the general case, we have seen that keeping data organized by both criteria will likely produce an unbalanced tree (which means, linear-time operations).

But that's not why treaps were invented, nor it's the main way they are used today.

In the end, the point is that there are better ways to index multidimensional data, and better ways use treaps.

We'll see, instead, that we can use treaps as a building block to implement a different, and efficient, data structure.

3.4.1 Balanced Trees

One aspect that we stressed is that unbalanced treaps tend to have long paths, whose length can be, in the worst-case scenario, in the order of $O(n)$ nodes.

Conversely, when discussing heaps, we saw that balanced trees, like indeed heaps, have logarithmic height, making all operations particularly convenient to run.

With heaps, however, the catch is that we trade the benefit of balanced trees in exchange to restricting to a limited set of operations: we can't efficiently search a heap for an element, nor retrieve its maximum or minimum, and even deleting or updating a random element (without knowing beforehand its position in the heap) require linear time.

Nevertheless, there are, in algorithms literature, many other examples of balanced trees, data structures that guarantees that the height of the tree will be logarithmic, even in the worst case.